CODE SUMMARIZATION

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Guideline 4: Do Not Trust Surface Fluency as a Quality Signal

- Judge a summary by what it lets a reader do, that is answer a question about the code, spot a bug, decide whether to reuse a function and not by how fluent or professional it sounds. Fluent summaries routinely miss key facts.

Reasoning

- Participants with human-written summaries performed significantly better on comprehension tasks; BLEU and ROUGE did not correlate with actual comprehension.
- LLM confidence scores are poorly calibrated: two summaries with nearly identical high confidences can differ greatly in accuracy.
- Neither fluency nor model confidence is a safe proxy for usefulness; both can be high while the summary omits critical inputs, outputs, or error conditions.

Key Practices: Prompting for Usefulness, Not Polish

- Tell the model the task the summary must support (e.g., 'a new maintainer deciding whether to modify this function') so output is shaped by the task, not by generic style.
- Ask for concrete, checkable facts (inputs, outputs, error modes, side effects) rather than adjectives.
- Prefer short, dense summaries over long polished ones, because length hides omissions.

Key Practices: Reviewing the Result

- Run a quick comprehension test: close the code and try to answer one concrete question using only the summary. If you cannot, the summary is not doing its job.
- Ignore the model's confidence language ('clearly', 'simply', 'robustly') when deciding whether to keep the summary.
- Compare the summary against the actual function signature and a sample call; missing parameters or error paths mean the summary is incomplete, no matter how well it reads.

Guideline 5: Adapt Your Prompt to the Language and Paradigm

- Change how you prompt based on the programming language and paradigm of the code. AI assistants perform noticeably worse on logic-programming code than on mainstream imperative code, and performance varies even between closely related languages. Compensate with explicit paradigm framing and worked examples.

Reasoning

- All four evaluated LLMs performed worst on logic-programming languages (e.g., Prolog); performance varied substantially within paradigms too.
- Paradigm-dependent performance reversals occurred between Haskell and Erlang, showing that language choice matters even within the functional paradigm.
- The LLM-as-judge evaluation protocol aligned well with human evaluation, giving students a practical way to cross-check their own summaries.

Key Practices: Prompting by Language and Paradigm

- Name the language and paradigm explicitly ('this is a Prolog predicate', 'this is a pointfree Haskell function') so the model does not default to imperative vocabulary.
- For logic, functional, or declarative code, include the meaning you want captured (unification pattern, fold structure, rewrite rule) rather than asking for a generic description.
- For less common languages, give a one-line behavioural anchor ('when queried with parent(tom, X), this returns all X such that...') to stabilise the output.

Key Practices: Reviewing the Result

- Check that the summary uses paradigm-appropriate vocabulary ('unifies', 'folds over', 'pattern-matches') rather than being translated into imperative steps.
- Be extra skeptical on languages or dialects where the model's training coverage is likely thinner.
- When unsure, ask the model to summarize the same code twice in two different ways and compare; wide disagreement signals you need more context in the prompt.

Guideline 6: Seed Project Style with a Few Local Exemplars

- When asking for a summary inside an existing project, include two or three example summaries from the same codebase so the AI picks up the project's conventions for terminology, length, voice, and level of detail. Do not rely on zero-shot to match project style.

Reasoning

- Few-shot prompting with exemplars from the same project significantly improved quality over both zero-shot and cross-project few-shot prompting.
- In-context exemplars let LLMs generate comments in distinct intent styles matching what developers in that project actually write.
- Vocabulary and conventions in real codebases are highly project-local; same-project exemplars are the cheapest way to transfer that dialect.

Key Practices: Prompting with Project Exemplars

- Pick two or three existing summaries from the same module or closely related modules; prefer summaries written by the project's most consistent author.
- Paste each exemplar as a (code, summary) pair in the same format you want the output in, with no extra commentary between them.
- Put the function to summarize last, so the model uses the exemplars as a style template and not the other way around.

Key Practices: Reviewing the Result

- Compare the output's length, tone, and terminology against the exemplars; large deviations usually mean the exemplars were ignored or were inconsistent with each other.
- Watch for generic phrasing ('This function is responsible for...') creeping back in; that is a sign the model fell back to its default style.
- If a project uses unusual conventions (Google-style docstrings, present-tense third-person), make sure at least one exemplar demonstrates that convention clearly.

References (1 of 2)

- [1] L. Pascarella and A. Bacchelli. "Classifying Code Comments in Java Open-Source Software Systems." 14th International Conference on Mining Software Repositories (MSR), 2017. DOI: 10.1109/MSR.2017.63.
- [2] M. Geng et al. "Large Language Models are Few-Shot Summarizers: Multi-Intent Comment Generation via In-Context Learning." ICSE 2024, pp. 453-465. arXiv:2304.11384. DOI: 10.1145/3597503.3608134.
- [3] T. Ahmed et al. "Automatic Semantic Augmentation of Language Model Prompts (for Code Summarization)." ICSE 2024. arXiv:2304.06815. DOI: 10.1145/3597503.3639183.
- [4] G. Sridhara et al. "Icing on the Cake: Automatic Code Summarization at Ericsson." ICSME Industry Track, 2024. arXiv:2408.09735.
- [5] S. Kang et al. "Identifying Inaccurate Descriptions in LLM-generated Code Comments via Test Execution." 2024. arXiv:2406.14836.

References (2 of 2)

- [6] S. Stapleton et al. "A Human Study of Comprehension and Code Summarization." ICPC 2020, pp. 2-13. DOI: 10.1145/3387904.3389258.
- [7] Y. Virk et al. "Calibration of Large Language Models on Code Summarization." Proc. ACM Softw. Eng., Vol. 2, FSE 2025. arXiv:2404.19318. DOI: 10.1145/3729400.
- [8] W. Sun et al. "Source Code Summarization in the Era of Large Language Models." ICSE 2025. arXiv:2407.07959.
- [9] T. Ahmed and P. Devanbu. "Few-shot Training LLMs for Project-Specific Code-Summarization." ASE NIER Track, 2022. arXiv:2207.04237. DOI: 10.1145/3551349.3559555.